# Machine Learning & Deep Learning

## Interview Questions and Answers

### Part 1 — Machine Learning

Fundamentals • Statistics • Classical ML • Evaluation • Deep Learning • CNNs • RNNs • Transformers • NLP • MLOps • Scenarios • Coding

# Table of Contents


1. Machine Learning Fundamentals
2. Statistics, Probability & Math
3. Data Preprocessing & Feature Engineering
4. Linear & Logistic Regression
5. Optimization
6. Decision Trees & Ensembles
7. SVM, KNN & Naive Bayes
8. Unsupervised Learning & Dimensionality Reduction
9. Model Evaluation & Metrics
10. Model Selection & Hyperparameter Tuning
11. Time Series
12. Recommender Systems

# Study Roadmap: Beginner to Advanced

Within every section, questions go roughly from basic to advanced. Suggested order:

| Level | Focus | Sections |
|---|---|---|
| **1. Beginner** | Core vocabulary, types of ML, basic statistics, data cleaning, linear/logistic regression, decision trees, basic metrics, neural network basics, Python libraries | 1, 2 (first half), 3, 4, 6 (first half), 9 (first half), 13 (first half), 21 (metrics, pandas, linear regression) |
| **2. Intermediate** | Optimization, ensembles and boosting, SVM/KNN/Naive Bayes, clustering and PCA, tuning, time series, training deep nets, CNNs, RNNs, NLP | 5, 6, 7, 8, 10, 11, 13–16, 18, 21 (k-means, KNN, logistic regression, PyTorch loop) |
| **3. Advanced** | Statistical testing and causal inference, XGBoost internals, calibration and uncertainty, recommender and ranking systems, Transformers, MLOps, system design, implementing attention/NMS from scratch | 2 (second half), 9 (second half), 12, 17, 19, 20, 21 (attention, NMS, AUC) |

**Tip:** For each answer, practise explaining it in 30 seconds, then be ready to go one level deeper (the math, a trade-off, or a real example from your projects).

# 1. Machine Learning Fundamentals

### Q1. What is the difference between AI, machine learning and deep learning?

**AI** is the broad field of building systems that perform tasks requiring human intelligence (including rule-based systems). **ML** is a subset of AI where systems learn patterns from data instead of being explicitly programmed. **Deep learning** is a subset of ML that uses multi-layer neural networks to learn features automatically from raw data.

### Q2. What is machine learning? How does it differ from traditional programming?

ML is the study of algorithms that improve at a task through experience (data) rather than explicit rules. In traditional programming, humans write rules: *data + rules → output*. In ML, the algorithm learns rules: *data + output → rules (model)*.

### Q3. What are features, labels and samples? Structured vs unstructured data?

A **feature** is an input variable (age, pixel value, word count); a **label/target** is what we want to predict; a **sample/instance** is one row/example, represented as a **feature vector**. **Structured** data is tabular (rows and columns, databases, spreadsheets). **Unstructured** data has no fixed schema (text, images, audio, video). **Semi-structured** data has some structure (JSON, XML, logs).

### Q4. What is the difference between training and inference?

**Training** learns model parameters from historical data; it is compute heavy and usually done offline. **Inference** (prediction/scoring) applies the trained model to new data; latency, throughput and cost matter most here.

### Q5. Give real-world applications of machine learning.

Spam filtering, product/movie recommendations, fraud detection, credit scoring, demand and sales forecasting, customer churn prediction, medical image diagnosis, speech recognition, machine translation, face recognition, self-driving car perception, predictive maintenance, search ranking, and dynamic pricing.

### Q6. What are the main types of machine learning?

- **Supervised**: labeled data; classification and regression.
- **Unsupervised**: unlabeled data; clustering, dimensionality reduction, density estimation.
- **Semi-supervised**: few labels + many unlabeled samples (pseudo-labeling, consistency regularization).
- **Self-supervised**: labels derived from data itself (masked tokens, next token, contrastive views).
- **Reinforcement learning**: agent learns via rewards from interacting with an environment.

### Q7. Classification vs regression?

Classification predicts discrete categories (spam / not spam); regression predicts continuous values (house price). Some problems can be framed either way (e.g., ordinal ratings).

### Q8. Explain the bias-variance trade-off.

Expected error = $\text{Bias}^2$ + Variance + Irreducible noise. **Bias** is error from overly simple assumptions (underfitting). **Variance** is sensitivity to fluctuations in training data (overfitting). Increasing model complexity lowers bias but raises variance; the goal is the sweet spot minimizing total error. Regularization, more data, and ensembling help manage it (bagging reduces variance; boosting reduces bias).

### Q9. What are overfitting and underfitting? How do you detect and fix them?

**Overfitting**: low training error, high validation error. Fix: more data, augmentation, regularization (L1/L2, dropout), simpler model, early stopping, feature selection, ensembling, cross-validation.
**Underfitting**: high training and validation error. Fix: more complex model, more/better features, less regularization, train longer.
Detect via learning curves (train vs validation error as data/epochs grow).

### Q10. Why split data into train, validation and test sets?

Train: fit parameters. Validation: tune hyperparameters / model selection / early stopping. Test: final unbiased estimate of generalization, touched only once. Using the test set for tuning leaks information and gives optimistic estimates. Typical splits: 70/15/15 or 80/10/10; for big data, 98/1/1 is fine.

### Q11. What is cross-validation? Describe its variants.

- **k-fold**: split into k folds; train on k-1, validate on 1, rotate; average scores.
- **Stratified k-fold**: preserves class proportions in each fold (important for imbalance).
- **Leave-one-out (LOOCV)**: k = n; low bias, high variance, expensive.
- **Group k-fold**: same group (e.g., patient/user) never appears in both train and validation.
- **Time-series split (forward chaining)**: train on past, validate on future only.
- **Nested CV**: inner loop tunes hyperparameters, outer loop estimates performance.

### Q12. What is data leakage? Give examples.

When information not available at prediction time leaks into training, producing overly optimistic results. Examples: scaling/imputing using statistics of the full dataset before splitting; features derived from the target (e.g., "account_closed_date" when predicting churn); future data in time series; duplicate records across train/test; same patient in train and test. Prevent with pipelines fit only on training folds, time-aware splits, group splits, and careful feature audits.

### Q13. What is the curse of dimensionality?

As dimensions grow, data becomes sparse, volume grows exponentially, distances become less discriminative (nearest and farthest neighbors become nearly equidistant), and you need exponentially more data to cover the space. It hurts distance-based methods (KNN, k-means). Mitigate with feature selection, dimensionality reduction (PCA), regularization, and domain knowledge.

### Q14. Parametric vs non-parametric models?

**Parametric**: fixed number of parameters regardless of data size (linear/logistic regression, Naive Bayes, neural nets of fixed size); strong assumptions, fast, less data needed. **Non-parametric**: complexity grows with data (KNN, decision trees, kernel SVM, Gaussian processes); flexible but needs more data and can overfit.

### Q15. Generative vs discriminative models?

Discriminative models learn P(y|x) or a decision boundary directly (logistic regression, SVM, neural classifiers). Generative models learn P(x, y) or P(x) and can generate samples (Naive Bayes, GMM, HMM, VAE, GAN, diffusion, LLMs). Discriminative usually gives better classification accuracy with enough data; generative handles missing data, works with fewer samples, and can synthesize data.

### Q16. What is the No Free Lunch theorem?

Averaged over all possible problems, no learning algorithm outperforms another. Hence model choice must be guided by assumptions about the specific problem (inductive bias) and empirical validation.

### Q17. What is inductive bias?

The set of assumptions a learner uses to generalize beyond training data. Examples: linear regression assumes linearity; CNNs assume locality and translation equivariance; RNNs assume sequential structure; KNN assumes nearby points share labels; Transformers have weak inductive bias and therefore need more data.

### Q18. What is the difference between a parameter and a hyperparameter?

Parameters are learned from data during training (weights, biases, split thresholds). Hyperparameters are set before training and control the learning process (learning rate, number of trees, depth, regularization strength, batch size, k in KNN).

### Q19. What is the difference between a loss function, a cost function and an objective function?

**Loss**: error for a single example (e.g., squared error). **Cost**: average loss over the dataset, often plus a regularization term. **Objective**: any function being optimized, minimized or maximized (e.g., maximizing likelihood). In practice the terms are often used interchangeably.

### Q20. What is a baseline model and why build one first?

A simple reference model: majority class, mean/median prediction, naive forecast (last value), simple rules, or plain logistic/linear regression. It sets the minimum bar, shows whether ML adds value, justifies extra complexity, and helps catch bugs or leakage (a complex model that barely beats, or suspiciously crushes, the baseline needs investigation).

### Q21. What is regularization and why is it used?

Any technique that constrains model complexity to reduce overfitting: L1/L2 penalties, dropout, early stopping, data augmentation, weight decay, pruning trees, max depth, label smoothing, noise injection. It trades a bit of bias for a large reduction in variance.

### Q22. What is the difference between online learning and batch learning?

Batch (offline) learning trains on the full dataset at once and is retrained periodically. Online learning updates the model incrementally as each sample/mini-batch arrives (SGD, streaming); useful for non-stationary data and data that does not fit in memory, but sensitive to noise and needs learning-rate control.

### Q23. Which Python libraries are commonly used for ML and DL?

**NumPy** (arrays, linear algebra), **pandas** (tabular data manipulation), **Matplotlib/Seaborn/Plotly** (visualization), **SciPy** and **statsmodels** (statistics, tests, classical models), **scikit-learn** (classical ML, preprocessing, pipelines, metrics), **XGBoost/LightGBM/CatBoost** (boosting), **PyTorch** and **TensorFlow/Keras** (deep learning), **OpenCV** (vision), **NLTK/spaCy/Hugging Face Transformers** (NLP), **MLflow** (experiment tracking).

### Q24. How would you approach a new ML problem end to end?

1. Understand the business goal and define success metrics (business + ML metric).
2. Check if ML is needed; establish a simple baseline/heuristic.
3. Collect data, understand labels, check quality, EDA.
4. Split data correctly (time/group aware) to avoid leakage.
5. Feature engineering and preprocessing in a pipeline.
6. Train simple models first, then more complex ones; tune with CV.
7. Error analysis; iterate on data and features.
8. Evaluate on held-out test; check fairness, robustness, latency.
9. Deploy, monitor drift and performance, retrain.

### Q25. How do you choose which algorithm to use?

Consider the problem type (classification, regression, clustering), data size and type, need for interpretability, latency/memory limits, linearity of relationships, number of features and missing values. Rules of thumb: tabular → gradient boosting / random forest (logistic/linear regression as a baseline); images → CNNs/ViTs; text → Transformers (TF-IDF + linear model as a baseline); small data → simpler, regularized models; need explanations → linear models or shallow trees. Start simple and let cross-validation decide.

### Q26. What is semi-supervised learning? Name some techniques.

Learning from a small labeled set plus a large unlabeled set. Techniques: self-training / pseudo-labeling, co-training, consistency regularization (FixMatch, Mean Teacher), label propagation on graphs, and pretraining with self-supervision followed by fine-tuning.

### Q27. What is active learning?

The model chooses which unlabeled samples should be labeled next, typically the most informative (uncertainty sampling, query-by-committee, expected model change, diversity sampling). Reduces labeling cost.

### Q28. What is transfer learning?

Reusing knowledge from a model trained on a source task/domain for a target task: use pretrained weights as a feature extractor or fine-tune some/all layers. Very effective when target data is limited (ImageNet CNNs, BERT, LLMs).

### Q29. What is multi-task learning and meta-learning?

**Multi-task learning**: one model trained jointly on related tasks with shared representations, improving generalization via shared signal. **Meta-learning** ("learning to learn"): train across many tasks so the model adapts quickly to new tasks with few examples (MAML, prototypical networks). Related: few-shot and zero-shot learning.

## 2. Statistics, Probability & Math

### Q30. What are conditional, joint and marginal probability? Independent vs mutually exclusive events?

**Joint** P(A, B): both occur. **Marginal** P(A): sum/integrate the joint over other variables. **Conditional** P(A|B) = P(A, B)/P(B): probability of A given B occurred. **Independent** events: P(A, B) = P(A)P(B), so knowing one tells nothing about the other. **Mutually exclusive**: cannot occur together, P(A, B) = 0; such events (with non-zero probability) are therefore dependent.

### Q31. State Bayes' theorem and give an ML use.

`P(A|B) = P(B|A) · P(A) / P(B)`. Posterior = likelihood × prior / evidence. Used in Naive Bayes classifiers, Bayesian inference, MAP estimation, Bayesian optimization, spam filtering, and medical test interpretation.

### Q32. MLE vs MAP estimation?

**MLE** chooses parameters maximizing P(data | θ). **MAP** maximizes P(θ | data) ∝ P(data | θ)P(θ), i.e., MLE plus a prior. A Gaussian prior on weights gives L2 regularization; a Laplace prior gives L1. With infinite data, MAP converges to MLE.

### Q33. Why is minimizing MSE equivalent to MLE under Gaussian noise, and cross-entropy to MLE for classification?

If $y = f(x) + \varepsilon$, $\varepsilon \sim N(0, \sigma^2)$, the negative log-likelihood is proportional to $\Sigma(y - f(x))^2$, so MLE = least squares. For Bernoulli/categorical labels, the negative log-likelihood is exactly binary/categorical cross-entropy.

### Q34. Common probability distributions you should know?

Bernoulli (single binary trial), Binomial (# successes in n trials), Poisson (count of events per interval), Geometric, Uniform, Normal/Gaussian, Exponential (time between events), Beta (probabilities; conjugate to Bernoulli), Dirichlet (multinomial prior; used in LDA), Multinomial/Categorical, Student-t (heavier tails), Chi-squared, Log-normal.

### Q35. Mean vs median vs mode? What are variance, standard deviation, skewness and kurtosis?

**Mean**: average, sensitive to outliers. **Median**: middle value, robust; preferred for skewed data (income, house prices). **Mode**: most frequent value; useful for categorical data. **Variance**: average squared deviation from the mean; **standard deviation** is its square root, in the original units. **Skewness**: asymmetry (right-skewed: mean > median). **Kurtosis**: heaviness of tails / outlier proneness.

### Q36. What are the properties of the normal distribution?

Symmetric and bell-shaped; mean = median = mode; fully defined by μ and σ. Empirical rule: ~68% of data within 1σ, ~95% within 2σ, ~99.7% within 3σ. Standardize with $z = (x - \mu)/\sigma$. Check normality with histograms, Q-Q plots, or Shapiro-Wilk / Kolmogorov-Smirnov tests.

### Q37. What are common sampling techniques?

**Simple random** (every item equally likely), **stratified** (sample within subgroups to preserve proportions; used for train/test splits on imbalanced data), **cluster** (randomly select whole groups), **systematic** (every k-th item), and **convenience** sampling (easy but biased). Over/undersampling adjusts class balance.

### Q38. Population vs sample? Standard deviation vs standard error?

The population is the full group of interest; a sample is a subset used to estimate population parameters (sample variance divides by $n - 1$ for an unbiased estimate). **Standard deviation** measures spread of individual data points. **Standard error** measures the uncertainty of an estimate such as the sample mean: $SE = \sigma/\sqrt{n}$, shrinking as sample size grows; it is used to build confidence intervals and test statistics.

### Q39. What is the Central Limit Theorem and why does it matter?

The distribution of the sample mean of i.i.d. variables with finite variance approaches a normal distribution as sample size grows, regardless of the original distribution. It justifies confidence intervals, z/t-tests, and A/B test statistics.

### Q40. Law of large numbers?

As the number of i.i.d. samples grows, the sample average converges to the expected value. Underlies Monte Carlo estimation and the use of empirical risk to approximate expected risk.

### Q41. Explain hypothesis testing, p-value, Type I and Type II errors, and statistical power.

Null hypothesis H0 (no effect) vs alternative H1. **p-value**: probability of observing data at least as extreme as seen, assuming H0 is true. Reject H0 if $p < \alpha$ (e.g., 0.05). **Type I error** (false positive): rejecting a true H0, rate $\alpha$. **Type II error** (false negative): failing to reject a false H0, rate $\beta$. **Power** $= 1 - \beta$, increases with sample size and effect size.

### Q42. When do you use a z-test, t-test, chi-square test or ANOVA?

- **z-test**: compare means with known population variance or large samples; compare proportions (A/B conversion rates).
- **t-test**: compare means with unknown variance / small samples; one-sample, two-sample (independent) or paired.
- **Chi-square test**: categorical data; goodness-of-fit or independence between two categorical variables (also used for feature selection).
- **ANOVA**: compare means across 3+ groups (F-test).
- Non-parametric alternatives when normality fails: Mann-Whitney U, Wilcoxon signed-rank, Kruskal-Wallis.

### Q43. What is a confidence interval?

A range computed from data such that, under repeated sampling, X% of such intervals contain the true parameter. It is not the probability the parameter lies in this particular interval (that is a Bayesian credible interval).

### Q44. How do you design and analyze an A/B test?

Define the hypothesis and primary metric; choose $\alpha$, power, minimum detectable effect; compute sample size; randomize users (unit of randomization); run for full business cycles; avoid peeking (or use sequential testing); check sample ratio mismatch; analyze with t-test / z-test for proportions / bootstrap; account for multiple comparisons (Bonferroni, FDR); consider novelty effects and network effects.

### Q45. Correlation vs causation? Pearson vs Spearman?

Correlation measures association; causation means changing X changes Y. Confounders, reverse causality, and selection bias can create correlation without causation; establish causality with randomized experiments or causal inference (instrumental variables, diff-in-diff, propensity scores). **Pearson** measures linear correlation; **Spearman** measures monotonic rank correlation and is robust to outliers.

### Q46. Covariance vs correlation?

Covariance measures joint variability and depends on units/scale. Correlation is covariance normalized by the product of standard deviations, bounded in [−1, 1] and unitless.

### Q47. What are entropy, cross-entropy and KL divergence?

**Entropy** $H(p) = -\Sigma\, p \log p$: uncertainty of a distribution. **Cross-entropy** $H(p, q) = -\Sigma\, p \log q$: expected bits to encode p using q. **KL divergence** $KL(p||q) = \Sigma\, p \log(p/q) = H(p, q) - H(p) \geq 0$; asymmetric, measures how q differs from p. Minimizing cross-entropy w.r.t. q is equivalent to minimizing KL since H(p) is constant. JS divergence is a symmetric, bounded version.

### Q48. What is mutual information?

$I(X;Y) = H(X) - H(X|Y)$: reduction in uncertainty about X from knowing Y. Zero iff independent; captures non-linear dependencies. Used for feature selection and in decision trees (information gain).

### Q49. What is the bootstrap?

Resampling the dataset with replacement many times to estimate the sampling distribution of a statistic (confidence intervals, standard errors). Also the basis of bagging. About 63.2% of unique samples appear in each bootstrap sample.

### Q50. Key linear algebra concepts for ML?

Vectors, matrices, dot product (similarity), matrix multiplication, transpose, inverse, rank, eigenvalues/eigenvectors (PCA, stability), SVD (PCA, recommender systems, low-rank approximation, LoRA), positive (semi-)definite matrices (covariance, convexity via Hessian), norms (L1, L2, Frobenius), orthogonality, projections.

### Q51. What are eigenvalues and eigenvectors? Where are they used in ML?

For a square matrix A, an eigenvector v is a non-zero vector whose direction is unchanged by A: $Av = \lambda v$, where the eigenvalue λ is the scaling factor. In PCA, eigenvectors of the covariance matrix are the principal directions and eigenvalues are the variance along them. Also used in spectral clustering, PageRank, analyzing Hessians (convexity) and RNN stability.

### Q52. Key calculus concepts for ML?

Derivatives and partial derivatives, gradient (direction of steepest ascent), chain rule (backpropagation), Jacobian (vector-valued functions), Hessian (curvature; second-order optimization, convexity check), Taylor expansion (justifies gradient descent and Newton's method).

### Q53. What is a convex function and why does it matter?

f is convex if the line segment between any two points lies above the graph (Hessian PSD). Any local minimum is global, so gradient methods converge to the optimum. Linear/logistic regression and SVM objectives are convex; neural network losses are non-convex.

### Q54. What is the difference between probability and likelihood?

Probability: parameters fixed, data varies; $P(\text{data} \mid \theta)$. Likelihood: data fixed, viewed as a function of parameters; $L(\theta \mid \text{data})$. Likelihood is not a probability distribution over θ.

### Q55. What is the i.i.d. assumption and why can it fail?

Independent and identically distributed: samples are drawn independently from the same distribution. It fails for time series (autocorrelation), grouped data (same user), and distribution shift between train and production, which invalidates naive CV and generalization estimates.

### Q56. What are Simpson's paradox and selection bias?

**Simpson's paradox**: a trend appearing in groups reverses when groups are combined, due to a confounder. **Selection bias**: the sample is not representative of the population (survivorship bias, non-response), causing models to fail in production.

### Q57. How do you estimate a causal effect when an A/B test isn't possible? What is uplift modeling?

Randomized experiments are the gold standard. With observational data use: propensity score matching/weighting (balance treated and control on observed covariates), difference-in-differences (compare changes over time against a control group), regression discontinuity (units just above vs below a cutoff), instrumental variables, and synthetic control. All rely on assumptions such as no unmeasured confounding. **Uplift modeling** predicts the *incremental* effect of a treatment for each individual, P(y | treated, x) − P(y | control, x), using T-/S-/X-learners or causal forests, so you target "persuadable" customers rather than those who would convert anyway.

### Q58. What is survival analysis?

Modeling time until an event (churn, machine failure, death) when some observations are **censored** (the event hasn't happened yet by the end of observation). Tools: Kaplan-Meier estimator (survival curve), log-rank test (compare groups), Cox proportional hazards model (hazard ratios for features; assumes proportional hazards), and ML variants such as random survival forests and XGBoost with AFT/Cox objectives. Evaluated with the concordance index (C-index).

## 3. Data Preprocessing & Feature Engineering

### Q59. What steps do you follow in Exploratory Data Analysis (EDA)?

1. Understand the dataset: shape, data types, meaning of columns and the target.
2. Summary statistics; missing values; duplicates; inconsistent or invalid values.
3. Univariate analysis: distributions (histograms, box plots), skewness, outliers, class balance.
4. Bivariate/multivariate analysis: correlation heatmap, scatter plots, feature vs target relationships, categorical cardinality.
5. Time trends and seasonality if data is temporal.
6. Flag possible leakage features and form hypotheses for feature engineering.

### Q60. How do you handle missing values?

First understand the mechanism: MCAR (completely at random), MAR (depends on observed data), MNAR (depends on the missing value itself). Options: drop rows/columns (if few or useless), mean/median/mode imputation, constant + "missing" indicator flag, KNN imputation, iterative/MICE (model-based), forward/backward fill for time series, or use models that handle NaNs natively (XGBoost, LightGBM). Fit imputers on training data only.

### Q61. Normalization vs standardization? When is scaling needed?

**Min-max normalization**: $(x - \min)/(\max - \min) \rightarrow [0, 1]$; sensitive to outliers. **Standardization (z-score)**: $(x - \mu)/\sigma \rightarrow$ mean 0, std 1. **Robust scaling**: uses median and IQR. Scaling matters for gradient-based models, distance-based methods (KNN, k-means, SVM), PCA and regularized models. Tree-based models are scale-invariant.

### Q62. How do you encode categorical variables?

- **One-hot**: nominal, low cardinality.
- **Label/ordinal encoding**: ordinal categories, or tree models.
- **Target/mean encoding**: high cardinality; use out-of-fold + smoothing to avoid leakage.
- **Frequency/count encoding**.
- **Hashing trick**: very high cardinality, fixed dimension.
- **Embeddings**: learned dense vectors in neural nets.
- CatBoost uses ordered target statistics natively.

### Q63. What is the dummy variable trap?

With one-hot encoding of k categories plus an intercept, the k columns always sum to 1, causing perfect multicollinearity in linear models. Fix: use $k - 1$ dummy columns (`drop_first=True`). Not an issue for tree models and less of one with regularization.

### Q64. How do you detect and handle outliers?

Detect: box plots / IQR rule (outside $Q1 - 1.5 \cdot IQR$, $Q3 + 1.5 \cdot IQR$), z-score > 3, Mahalanobis distance, isolation forest, LOF, DBSCAN. Handle: verify if error or genuine; remove, cap/winsorize, log/Box-Cox transform, use robust models/losses (Huber, MAE, trees), or treat as separate signal (fraud).

### Q65. How do you handle imbalanced datasets?

- Use proper metrics: precision, recall, F1, PR-AUC instead of accuracy.
- Resampling: random oversampling, undersampling, SMOTE/ADASYN (on training data only).
- Class weights / cost-sensitive learning (`class_weight='balanced'`, `scale_pos_weight`).
- Threshold tuning on the validation set.
- Focal loss for deep learning; ensembles like balanced random forest / EasyEnsemble.
- Anomaly detection framing when positives are extremely rare.
- Collect more minority data.

### Q66. What is SMOTE?

Synthetic Minority Over-sampling Technique: for a minority sample, pick one of its k nearest minority neighbors and create a synthetic point on the line segment between them. Reduces overfitting vs duplicate oversampling, but can create noise near class boundaries and doesn't work well for high-dimensional/categorical data. Apply only inside training folds.

### Q67. What are feature selection methods?

- **Filter**: statistical scores independent of model (correlation, chi-square, ANOVA F-test, mutual information, variance threshold).
- **Wrapper**: search subsets using model performance (forward selection, backward elimination, RFE).
- **Embedded**: selection during training (L1/Lasso, tree feature importance).

Benefits: less overfitting, faster training, interpretability.

### Q68. Feature selection vs feature extraction?

**Selection** keeps a subset of the original features, preserving interpretability (Lasso, RFE, mutual information). **Extraction** creates new features by transforming or combining existing ones (PCA, LDA, autoencoders, embeddings); can capture more information in fewer dimensions but new features are harder to interpret.

### Q69. What is multicollinearity and how do you detect and handle it?

High correlation among features. It inflates coefficient variance in linear models, making coefficients unstable and hard to interpret (predictions may still be fine). Detect with correlation matrix and Variance Inflation Factor (VIF > 5–10). Handle by dropping/combining features, PCA, or Ridge regularization.

### Q70. Why apply log or power transforms?

To reduce right skew, stabilize variance, make relationships more linear, and lessen outlier impact. Examples: log(1+x), square root, Box-Cox (positive data), Yeo-Johnson (any sign). Also used on targets like price, then exponentiated back.

### Q71. What feature engineering techniques do you know?

Binning/discretization, polynomial and interaction features, ratios and differences, aggregations (group-by mean/count/std per user), date-time features (hour, weekday, holiday, cyclical sin/cos encoding), lag and rolling-window features, text features (TF-IDF, embeddings), geospatial features (distance, clusters), domain-specific features, target encoding.

### Q72. Why use cyclical encoding for time features?

Hour 23 and hour 0 are adjacent but numerically far. Encoding as $\sin(2\pi \cdot h/24)$ and $\cos(2\pi \cdot h/24)$ preserves circular continuity.

### Q73. How should preprocessing be applied to train and test sets?

Fit transformers (scalers, imputers, encoders, PCA) on training data only, then transform validation/test with those fitted parameters. Use pipelines (e.g., scikit-learn `Pipeline` / `ColumnTransformer`) so the same steps run in each CV fold and in production.

### Q74. What is the difference between fit, transform and fit_transform?

`fit` learns parameters from data (e.g., mean and std for a scaler, vocabulary for a vectorizer). `transform` applies the learned parameters to data. `fit_transform` does both in one step and should be used only on training data; validation, test and production data must use `transform` only, otherwise information leaks.

### Q75. How do you deal with high-cardinality categorical features?

Target encoding with smoothing and out-of-fold computation, frequency encoding, grouping rare categories into "Other", hashing, learned embeddings, or CatBoost's native handling. Avoid one-hot explosion.

### Q76. What is data augmentation? Give examples across modalities.

Creating modified copies of data to increase diversity and reduce overfitting. Images: flips, crops, rotation, color jitter, cutout, mixup, cutmix. Text: synonym replacement, back-translation, random deletion, paraphrasing with LLMs. Audio: time-shift, pitch shift, noise, SpecAugment. Tabular: SMOTE, noise injection.

### Q77. How do you handle noisy labels?

Label auditing and relabeling, consensus of multiple annotators, confident learning (cleanlab) to find mislabeled samples, robust losses (MAE, symmetric CE), label smoothing, co-teaching, early stopping (nets memorize noise late).

## 4. Linear & Logistic Regression

### Q78. Explain linear regression.

Models the target as a linear combination of features: $\hat{y} = \beta_0 + \beta_1 x_1 + \ldots + \beta_p x_p$. Parameters are fit by minimizing the sum of squared residuals (Ordinary Least Squares), via the normal equation or gradient descent. Simple regression has one feature; multiple regression has many. Fast, interpretable, and a strong baseline; evaluated with RMSE, MAE and $R^2$.

### Q79. What are the assumptions of linear regression?

1. Linearity between features and target.
2. Independence of errors.
3. Homoscedasticity (constant error variance).
4. Normality of residuals (for inference).
5. No (perfect) multicollinearity.

Check with residual plots, Q-Q plots, Durbin-Watson test, VIF.

### Q80. What is the closed-form (normal equation) solution of linear regression? When not to use it?

$\beta = (X^T X)^{-1} X^T y$. Cost is $O(n \cdot d^2 + d^3)$; impractical for many features or when $X^T X$ is singular (multicollinearity, $d > n$). Use gradient descent, QR/SVD, or Ridge, which gives $(X^T X + \lambda I)^{-1} X^T y$ and is always invertible.

### Q81. L1 (Lasso) vs L2 (Ridge) vs Elastic Net?

| | L1 (Lasso) | L2 (Ridge) |
|---|---|---|
| Penalty | $\lambda \Sigma \lvert w \rvert$ | $\lambda \Sigma w^2$ |
| Effect | Sparse weights (feature selection) | Shrinks weights smoothly, none exactly zero |
| Correlated features | Picks one arbitrarily | Spreads weight among them |
| Prior | Laplace | Gaussian |

Elastic Net combines both: good with correlated features while still being sparse. L1 creates sparsity because its diamond-shaped constraint region has corners on the axes where the loss contours tend to touch.

### Q82. Explain $R^2$ and adjusted $R^2$.

$R^2 = 1 - SS_{res}/SS_{tot}$: fraction of target variance explained. It never decreases when adding features. Adjusted $R^2 = 1 - (1 - R^2)(n - 1)/(n - p - 1)$ penalizes extra features. $R^2$ can be negative on test data if the model is worse than predicting the mean.

### Q83. How do you check whether a feature is significant in linear regression?

Each coefficient has a t-statistic ($\beta/SE(\beta)$) and p-value testing H0: $\beta = 0$; $p < 0.05$ suggests the feature is significant. The F-test checks whether the model as a whole (or a group of features) is significant. Confidence intervals that exclude 0 tell the same story. Multicollinearity inflates standard errors and can hide significant features.

### Q84. How do you interpret coefficients in linear regression?

$\beta_j$ is the expected change in y for a one-unit increase in $x_j$, holding other features constant. With standardized features, magnitudes are comparable. With log target, coefficients approximate percentage change. Multicollinearity makes interpretation unreliable.

### Q85. What is logistic regression? Why is it called regression?

A linear classifier modeling the log-odds as linear: $\log(p/(1-p)) = w^T x + b$, so $p = \sigma(w^T x + b)$ with the sigmoid $\sigma(z) = 1/(1+e^{-z})$. It "regresses" the probability (log-odds) and is a GLM with a logit link. The decision boundary is linear.

### Q86. Linear regression vs logistic regression? Why not use linear regression for classification?

| | **Linear regression** | **Logistic regression** |
|---|---|---|
| Task | Regression (continuous target) | Classification (probability of class) |
| Output | Unbounded real value | Probability in (0, 1) via sigmoid |
| Loss | Mean squared error | Log loss (cross-entropy) |
| Estimation | OLS / closed form | MLE via iterative optimization |
| Metrics | RMSE, MAE, $R^2$ | Accuracy, precision, recall, AUC, log loss |

Linear regression for classification produces values outside [0, 1], and outliers shift the fitted line and therefore the decision threshold.

### Q87. What is the loss function of logistic regression and why not MSE?

Binary cross-entropy (log loss): $-[y \log p + (1-y) \log(1-p)]$. It is the Bernoulli negative log-likelihood and is convex in w. MSE with sigmoid is non-convex and yields small gradients when predictions are confidently wrong.

### Q88. How do you interpret logistic regression coefficients?

A one-unit increase in $x_j$ changes the log-odds by $\beta_j$, i.e., multiplies the odds by $e^{\beta_j}$ (odds ratio).

### Q89. How does logistic regression handle multi-class problems?

One-vs-Rest (train K binary classifiers) or multinomial (softmax) regression: $p_k = e^{z_k}/\Sigma e^{z_j}$, trained with categorical cross-entropy.

### Q90. What happens with perfectly separable data in logistic regression?

The MLE does not exist: weights grow to infinity to push probabilities to 0/1. Regularization fixes this.

### Q91. What are polynomial regression and GLMs?

Polynomial regression adds powers/interactions of features and is still linear in parameters. Generalized Linear Models connect a linear predictor to the mean of an exponential-family distribution through a link function: linear (identity, Gaussian), logistic (logit, Bernoulli), Poisson regression (log, counts), Gamma regression.

### Q92. What is heteroscedasticity and how to fix it?

Non-constant residual variance (e.g., errors grow with predicted value). Coefficients remain unbiased but standard errors are wrong. Fix with transforming the target (log), weighted least squares, or robust (Huber-White) standard errors.

### Q93. What are robust regression methods?

Methods less sensitive to outliers: Huber loss (quadratic near zero, linear far away), MAE / quantile regression, RANSAC, Theil-Sen.

## 5. Optimization

### Q94. Explain gradient descent.

Iteratively update parameters opposite to the gradient of the loss: $\theta \leftarrow \theta - \eta \nabla L(\theta)$. Learning rate $\eta$ too small → slow; too large → divergence/oscillation.

### Q95. Batch vs stochastic vs mini-batch gradient descent?

**Batch GD**: full dataset per step; stable, slow, memory heavy. **SGD**: one sample per step; noisy, fast updates, noise can escape saddle points/sharp minima. **Mini-batch** (32–1024): compromise; uses GPU parallelism efficiently; the standard in deep learning.

### Q96. What is momentum? Nesterov momentum?

Momentum accumulates an exponentially decaying average of past gradients: $v = \beta v + \nabla L$; $\theta \mathrel{-}= \eta v$. It accelerates along consistent directions and dampens oscillations in ravines. Nesterov evaluates the gradient at the look-ahead position ($\theta - \eta\beta v$), giving more responsive corrections.

### Q97. Explain AdaGrad, RMSProp and Adam.

- **AdaGrad**: per-parameter learning rates scaled by the sum of squared past gradients; good for sparse features but learning rate decays to near zero.
- **RMSProp**: uses an exponential moving average of squared gradients instead of the sum, fixing AdaGrad's decay.
- **Adam**: momentum (1st moment m) + RMSProp (2nd moment v) with bias correction: $\theta \mathrel{-}= \eta \cdot \hat{m}/(\sqrt{\hat{v}} + \varepsilon)$. Defaults $\beta_1$=0.9, $\beta_2$=0.999. Fast and robust; sometimes generalizes slightly worse than SGD+momentum.

### Q98. What is AdamW and why is it preferred?

In Adam, L2 regularization added to the loss gets scaled by the adaptive denominator, so it is not true weight decay. AdamW decouples weight decay, applying it directly to weights ($\theta \mathrel{-}= \eta\lambda\theta$). It generalizes better and is the default for Transformers.

### Q99. What are learning rate schedules?

Step decay, exponential decay, cosine annealing, cosine with warm restarts (SGDR), one-cycle policy, reduce-on-plateau, linear warmup then decay (standard for Transformers since early large updates with Adam's unstable statistics can diverge).

### Q100. How do you choose a good learning rate?

Start from common defaults (Adam: 1e-3 or 3e-4; SGD with momentum: ~0.1 for CNNs), then tune on a log scale. Use an LR range test: increase the LR exponentially over a few hundred steps and pick a value about 10× below where the loss starts to blow up. Read loss curves: diverging or oscillating means too high; very slow decrease means too low. Combine with warmup and decay schedules.

### Q101. What are saddle points and local minima? Which is the bigger problem in deep learning?

In high dimensions, saddle points (gradient zero, curvature positive in some directions and negative in others) are far more common than bad local minima. Most local minima of large networks have similar loss. Momentum and SGD noise help escape saddle points and plateaus.

### Q102. What are second-order optimization methods? Why are they rarely used in deep learning?

Newton's method uses the Hessian: $\theta \mathrel{-}= H^{-1}\nabla L$; converges in fewer steps. Quasi-Newton methods (BFGS, L-BFGS) approximate the Hessian. For networks with millions of parameters, storing/inverting a d×d Hessian is infeasible, and stochastic mini-batches make curvature estimates noisy.

### Q103. How does batch size affect training?

Larger batches: more accurate gradients, better hardware utilization, fewer steps, but may converge to sharper minima and generalize worse and need learning-rate scaling (linear scaling rule with warmup). Smaller batches: noisier gradients acting as regularization, more updates, slower per epoch. Memory limits batch size; gradient accumulation simulates larger batches.

### Q104. What is gradient clipping?

Limiting gradient magnitude (by value or by global norm) before the update to prevent exploding gradients, especially in RNNs and Transformers.

### Q105. What is early stopping?

Monitor validation loss and stop when it hasn't improved for a "patience" number of epochs, restoring the best weights. Acts as regularization, similar in effect to L2 for linear models.

### Q106. What is the difference between an epoch, a batch and an iteration?

Epoch: one full pass over the training data. Batch: subset of samples used in one update. Iteration: one parameter update. Iterations per epoch = dataset size / batch size.

## 6. Decision Trees & Ensembles

### Q107. How does a decision tree work?

Recursively splits data on the feature/threshold that best improves purity (classification) or reduces variance (regression), forming a tree of if-else rules. Leaves predict the majority class or mean value. Greedy, top-down (CART, ID3, C4.5).

### Q108. Gini impurity vs entropy (information gain)?

Gini = $1 - \Sigma p_k^2$; Entropy = $-\Sigma p_k \log p_k$. Information gain = parent entropy − weighted child entropy. Both give similar trees; Gini is slightly faster (no log). Information gain is biased toward high-cardinality features; C4.5 uses gain ratio to correct this. Regression trees use MSE/variance reduction.

### Q109. How do you prevent overfitting in decision trees?

Pre-pruning: max_depth, min_samples_split, min_samples_leaf, max_leaf_nodes, min_impurity_decrease. Post-pruning: cost-complexity pruning (ccp_alpha). Or use ensembles (random forest, boosting).

### Q110. Advantages and disadvantages of decision trees?

Pros: interpretable, handle non-linearity and interactions, no scaling needed, handle mixed types, fast inference. Cons: high variance/unstable, overfit easily, axis-aligned splits, poor extrapolation, greedy (not globally optimal), biased toward dominant classes.

### Q111. What is bagging?

Bootstrap Aggregating: train models on bootstrap samples in parallel and average (regression) or vote (classification). Reduces variance; best with high-variance, low-bias base learners like deep trees.

### Q112. How does random forest work and why is it better than bagged trees?

Bagging of decision trees plus random feature subsampling at each split (typically $\sqrt{d}$ for classification, $d/3$ for regression). The feature randomness decorrelates trees, so averaging reduces variance more. Robust, little tuning, gives feature importance and OOB error.

### Q113. What is out-of-bag (OOB) error?

Each tree leaves out ~36.8% of samples. Predicting each sample using only trees that didn't see it gives an OOB estimate of generalization error, a free validation score without a separate holdout.

### Q114. What is boosting? How does it differ from bagging?

Boosting trains weak learners sequentially, each focusing on errors of the previous ensemble, and combines them in a weighted sum. Reduces bias (and variance). Bagging is parallel, independent models, reduces variance. Boosting is more prone to overfitting noise and needs careful tuning (learning rate, n_estimators, depth).

### Q115. Explain AdaBoost.

Start with equal sample weights; train a weak learner (often a stump); compute weighted error $\varepsilon$ and learner weight $\alpha = \frac{1}{2} \ln((1-\varepsilon)/\varepsilon)$; increase weights of misclassified samples; repeat. Final prediction: $\text{sign}(\Sigma \alpha_t h_t(x))$. Equivalent to minimizing exponential loss; sensitive to outliers.

### Q116. Explain gradient boosting.

Builds an additive model $F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x)$, where each new tree $h_m$ fits the negative gradient of the loss w.r.t. current predictions (pseudo-residuals; for MSE these are ordinary residuals). It's gradient descent in function space and works with any differentiable loss. Key hyperparameters: learning rate, n_estimators, max_depth, subsample, min_child_weight.

### Q117. What makes XGBoost better than vanilla gradient boosting?

Second-order Taylor approximation of the loss (gradient + Hessian), regularized objective (L1/L2 on leaf weights, γ penalty on number of leaves), sparsity-aware split finding (learns default direction for missing values), weighted quantile sketch for approximate splits, column/row subsampling, parallelized split finding, cache-aware and out-of-core computation, built-in CV and early stopping.

### Q118. How does XGBoost compute leaf weights and split gain?

Using a second-order Taylor expansion, each sample has gradient $g_i$ and Hessian $h_i$. For a leaf with sums $G = \Sigma g_i$ and $H = \Sigma h_i$, the optimal weight is `w* = −G / (H + λ)`. The gain of splitting a node into left and right children is:

$$\text{Gain} = \tfrac{1}{2}\left[\frac{G_L^2}{H_L+\lambda} + \frac{G_R^2}{H_R+\lambda} - \frac{(G_L+G_R)^2}{H_L+H_R+\lambda}\right] - \gamma$$

A split is kept only if the gain is positive, so γ acts as built-in pruning and λ shrinks leaf weights. `min_child_weight` sets a minimum H per leaf.

### Q119. XGBoost vs LightGBM vs CatBoost?

| XGBoost | LightGBM | CatBoost |
|---|---|---|
| Level-wise growth; histogram or exact splits; very mature. | Leaf-wise (best-first) growth: faster, lower loss, but can overfit small data (limit num_leaves). Histogram binning, GOSS (gradient-based one-side sampling), EFB (exclusive feature bundling). Fastest on large data. | Symmetric (oblivious) trees; native categorical handling with ordered target statistics; ordered boosting to reduce prediction shift/target leakage; strong defaults. |

### Q120. Random forest vs gradient boosting: differences and when to use which?

| | Random forest | Gradient boosting |
|---|---|---|
| Training | Trees built independently, in parallel | Trees built sequentially, each correcting previous errors |
| Reduces | Variance | Bias (and variance) |
| Trees | Deep, fully grown | Shallow (weak learners) |
| Tuning | Works well with defaults | Sensitive (learning rate, depth, n_estimators) |
| Overfitting | More trees don't overfit | Too many trees can overfit; use early stopping |
| Noise/outliers | More robust | More sensitive |

Use random forest for a quick, robust baseline; gradient boosting (XGBoost/LightGBM/CatBoost) for best accuracy on tabular data when you can tune.

### Q121. How is feature importance computed in tree models? What are its pitfalls?

Impurity-based (mean decrease in Gini/variance), split count, gain. Pitfalls: biased toward high-cardinality/continuous features, computed on training data, unreliable with correlated features. Better: permutation importance on validation data, or SHAP values (TreeSHAP).

### Q122. What are stacking, blending and voting?

**Voting**: combine predictions by majority (hard) or averaged probabilities (soft). **Stacking**: train a meta-model on out-of-fold predictions of base models. **Blending**: like stacking but the meta-model is trained on predictions on a holdout set. Works best with diverse base models.

### Q123. Why do ensembles work?

If base models are accurate and make uncorrelated errors, averaging cancels individual errors, reducing variance (and boosting reduces bias). Variance of an average of n models with correlation ρ is $\rho\sigma^2 + (1-\rho)\sigma^2/n$, so lowering correlation is key.

### Q124. Can tree-based models extrapolate?

No. Predictions are constants (averages) in leaves, so outside the training range they stay flat at boundary values. For trends (e.g., time series growth), detrend first or use linear models.

### Q125. Why do gradient boosted trees often beat deep learning on tabular data?

Tabular data has heterogeneous, non-smooth, often irregular features and moderate sample sizes; trees handle irregular decision boundaries, uninformative features, missing values, and different scales without preprocessing, and are easier to tune.

## 7. SVM, KNN & Naive Bayes

### Q126. Explain Support Vector Machines.

SVM finds the hyperplane that maximizes the margin between classes. The closest points (support vectors) define it. Hard margin requires separability; soft margin allows violations via slack variables controlled by C. Objective: min $\frac{1}{2}||w||^2 + C\Sigma\xi_i$, equivalently hinge loss $\max(0, 1 - y \cdot f(x))$ + L2 regularization.

### Q127. What is the effect of the C parameter in SVM?

Large C: penalizes misclassification heavily → narrow margin, low bias, high variance. Small C: wider margin, more violations allowed → more regularization.

### Q128. What is the kernel trick?

Kernels compute inner products in a high-dimensional feature space without explicitly mapping data: $K(x, z) = \varphi(x) \cdot \varphi(z)$. Since the SVM dual depends only on inner products, it can learn non-linear boundaries efficiently. Kernels: linear, polynomial $(x \cdot z + c)^d$, RBF/Gaussian $\exp(-\gamma||x-z||^2)$, sigmoid. Valid kernels must be positive semi-definite (Mercer's theorem).

### Q129. What does gamma do in the RBF kernel?

Controls the reach of each support vector. High γ: very local influence, wiggly boundary, overfitting. Low γ: smoother boundary, may underfit. Tune C and γ together.

### Q130. SVM pros and cons? Does SVM output probabilities?

Pros: effective in high dimensions, memory efficient (support vectors), strong with clear margin, flexible kernels. Cons: $O(n^2)$–$O(n^3)$ training (poor for large n), sensitive to scaling and hyperparameters, less interpretable. SVM outputs distances, not probabilities; use Platt scaling for calibrated probabilities. SVR is the regression version with an ε-insensitive tube.

### Q131. How does KNN work and how do you choose k?

Lazy, instance-based: predict by majority vote (classification) or average (regression) of the k nearest training points. Small k: low bias, high variance, noisy. Large k: smoother, more bias. Choose k via CV; use odd k for binary classification. Use distance weighting optionally.

### Q132. What distance metrics are used in ML?

Euclidean (L2), Manhattan (L1), Minkowski (generalization), Chebyshev (L∞), cosine (orientation; text/embeddings), Hamming (categorical/binary), Jaccard (sets), Mahalanobis (accounts for covariance).

### Q133. KNN drawbacks and speedups?

Slow prediction O(n·d) per query, memory heavy, needs scaling, suffers from curse of dimensionality, sensitive to irrelevant features and imbalance. Speedups: KD-trees, Ball trees, approximate nearest neighbor (HNSW, FAISS, LSH).

### Q134. How does Naive Bayes work? Why "naive"?

Applies Bayes' theorem assuming features are conditionally independent given the class: $P(y|x) \propto P(y)\Pi P(x_i|y)$. "Naive" because independence rarely holds, but it still classifies well (ranking is often correct even if probabilities are poorly calibrated). Variants: Gaussian (continuous), Multinomial (counts, text), Bernoulli (binary features), Complement NB (imbalanced text).

### Q135. What is Laplace smoothing?

Adding a pseudo-count (α, typically 1) to every feature count so unseen feature-class combinations don't get zero probability, which would zero out the whole product.

### Q136. Why compute Naive Bayes in log space?

Multiplying many small probabilities underflows to zero; summing log probabilities is numerically stable.

## 8. Unsupervised Learning & Dimensionality Reduction

### Q137. Explain k-means clustering.

Initialize k centroids; assign each point to its nearest centroid; recompute centroids as cluster means; repeat until convergence. Minimizes within-cluster sum of squares (inertia). Converges to a local optimum; complexity O(n·k·d·iterations).

### Q138. What is k-means++?

Smart initialization: choose the first centroid randomly, then pick subsequent centroids with probability proportional to squared distance from the nearest chosen centroid. Spreads centroids out, faster convergence and better results.

### Q139. How do you choose k in k-means?

Elbow method (inertia vs k), silhouette score (higher is better, range −1 to 1), gap statistic, Davies-Bouldin index (lower better), Calinski-Harabasz (higher better), and domain knowledge/business use.

### Q140. Limitations of k-means?

Must choose k; assumes spherical, similarly sized clusters; sensitive to initialization, outliers and scaling; only numeric data (use k-modes/k-prototypes for categorical); struggles with non-convex shapes.

### Q141. K-means vs KNN?

**K-means**: unsupervised clustering; k = number of clusters; iteratively learns centroids. **KNN**: supervised classification/regression; k = number of neighbors; lazy learner with no training phase. Both rely on distance metrics and need feature scaling.

### Q142. Explain hierarchical clustering.

Agglomerative (bottom-up): start with each point as a cluster and repeatedly merge the closest pair; divisive is top-down. Linkage: single (min distance; chaining), complete (max), average, Ward (minimizes variance increase). Produces a dendrogram; cut at a height to get clusters. $O(n^2)$ memory, no need to pre-specify k.

### Q143. Explain DBSCAN.

Density-based: points with at least minPts neighbors within ε are core points; clusters grow from core points through density-reachable points; others are noise. Finds arbitrary shapes, detects outliers, no k needed. Struggles with varying densities (use HDBSCAN) and high dimensions; sensitive to ε.

### Q144. What are Gaussian Mixture Models and the EM algorithm?

GMM models data as a mixture of K Gaussians with soft (probabilistic) assignments and elliptical clusters. Fit with Expectation-Maximization: **E-step** computes responsibilities (posterior cluster probabilities) given current parameters; **M-step** updates means, covariances, and mixing weights to maximize expected log-likelihood. K-means is a special case (hard assignments, equal spherical covariance). Choose K with BIC/AIC.

### Q145. Explain PCA.

Linear dimensionality reduction that projects data onto orthogonal directions (principal components) of maximum variance. Steps: standardize, compute covariance matrix, eigen-decompose (or SVD of centered data), sort eigenvectors by eigenvalue, project onto top k. Choose k by explained variance (e.g., 95%) or scree plot. Limitations: linear, sensitive to scaling, components less interpretable, variance ≠ predictive relevance.

### Q146. How are PCA and SVD related?

For centered data $X = U\Sigma V^T$, the columns of V are the principal directions, and eigenvalues of the covariance are $\sigma_i^2/(n-1)$. SVD is more numerically stable than forming the covariance matrix.

### Q147. PCA vs LDA (Linear Discriminant Analysis)?

PCA is unsupervised and maximizes variance. LDA is supervised and maximizes between-class separation relative to within-class scatter; at most $(C-1)$ components. LDA is also a classifier assuming Gaussian classes with shared covariance.

### Q148. t-SNE vs UMAP?

Both are non-linear methods for visualization. **t-SNE** matches neighbor probability distributions (Gaussian in high-D, Student-t in low-D to avoid crowding); preserves local structure, slow, stochastic, perplexity-sensitive; cluster sizes/distances are not meaningful; no transform for new data. **UMAP**: based on manifold theory and fuzzy topological graphs; faster, scales better, preserves more global structure, supports transforming new points.

### Q149. What other dimensionality reduction methods exist?

Kernel PCA (non-linear), ICA (statistically independent sources; blind source separation), NMF (non-negative parts-based; topic modeling), truncated SVD/LSA (sparse text), autoencoders, feature selection, random projections (Johnson-Lindenstrauss).

### Q150. What anomaly detection methods do you know?

Statistical (z-score, IQR, Gaussian density), Isolation Forest (anomalies are isolated in fewer random splits → shorter path length), Local Outlier Factor (local density compared to neighbors), One-Class SVM, DBSCAN noise points, autoencoder reconstruction error, and forecasting residuals for time series.

### Q151. What is association rule mining?

Finding item co-occurrence rules $X \rightarrow Y$ in transactions (market basket). Metrics: support = $P(X \cup Y)$, confidence = $P(Y|X)$, lift = confidence / $P(Y)$ ($>1$ means positive association). Algorithms: Apriori (prunes using frequent-subset property), FP-Growth (prefix tree, no candidate generation).

### Q152. How do you evaluate clustering without labels?

Internal metrics: silhouette, Davies-Bouldin, Calinski-Harabasz, inertia. With ground truth: Adjusted Rand Index, Normalized Mutual Information, homogeneity/completeness. Also stability across resamples and business usefulness.

### Q153. What is topic modeling (LDA - Latent Dirichlet Allocation)?

Generative model where each document is a mixture of topics and each topic is a distribution over words, with Dirichlet priors. Inferred via Gibbs sampling or variational inference. Evaluate with coherence scores. Modern alternative: BERTopic (embeddings + clustering + c-TF-IDF).

## 9. Model Evaluation & Metrics

### Q154. Explain the confusion matrix and derived metrics.

| | **Predicted +** | **Predicted −** |
|---|---|---|
| **Actual +** | TP | FN (Type II) |
| **Actual −** | FP (Type I) | TN |

Accuracy = (TP+TN)/All. Precision = TP/(TP+FP). Recall/Sensitivity/TPR = TP/(TP+FN). Specificity/TNR = TN/(TN+FP). FPR = FP/(FP+TN). F1 = 2PR/(P+R).

### Q155. When to prioritize precision vs recall?

**Precision** when false positives are costly: spam filtering (don't lose real emails), recommending content, legal decisions. **Recall** when false negatives are costly: cancer screening, fraud detection, safety defects. Fβ balances them (β > 1 weights recall).

### Q156. Why does the F1 score use the harmonic mean instead of the arithmetic mean?

The harmonic mean is dominated by the smaller value, so F1 is high only when both precision and recall are high. A model with precision 1.0 and recall 0.0 would get 0.5 with the arithmetic mean but 0 with F1.

### Q157. Why is accuracy misleading on imbalanced data?

With 99% negatives, predicting all negative gives 99% accuracy but zero recall. Use precision, recall, F1, PR-AUC, balanced accuracy, MCC (Matthews correlation coefficient), or Cohen's kappa.

### Q158. Explain ROC curve and AUC.

ROC plots TPR vs FPR across all thresholds. AUC is the probability that a random positive is scored higher than a random negative; 0.5 = random, 1.0 = perfect. Threshold-independent and scale-invariant; measures ranking quality.

### Q159. ROC-AUC vs PR-AUC?

With heavy imbalance, FPR stays small because TN is huge, so ROC-AUC can look optimistic. PR curves (precision vs recall) focus on the positive class and are more informative for rare positives; baseline PR-AUC equals positive prevalence.

### Q160. How do you choose a classification threshold?

Based on business costs: maximize F1/Fβ, use Youden's J (TPR − FPR) on ROC, set a minimum precision or recall constraint, or minimize expected cost ($cost_{FP}$·FP + $cost_{FN}$·FN). Choose on validation data, not test.

### Q161. Micro vs macro vs weighted averaging in multi-class metrics?

**Macro**: compute metric per class, average equally (treats rare classes equally). **Micro**: aggregate TP/FP/FN globally (dominated by frequent classes; micro-F1 = accuracy in single-label multi-class). **Weighted**: macro weighted by class support.

### Q162. Explain regression metrics.

- **MAE**: mean |error|; robust to outliers; median-optimal.
- **MSE**: mean squared error; penalizes large errors; mean-optimal; differentiable.
- **RMSE**: √MSE; same units as target.
- **$R^2$ / adjusted $R^2$**: variance explained.
- **MAPE**: mean |error/actual|; scale-free but undefined at zero and asymmetric; sMAPE fixes partly.
- **RMSLE**: penalizes under-prediction more; for targets spanning orders of magnitude.
- **Huber**: MSE for small errors, MAE for large.

### Q163. What is log loss and why use it?

$-(1/N)\Sigma[y \log p + (1-y) \log(1-p)]$. Evaluates the quality of predicted probabilities, heavily penalizing confident wrong predictions. Useful when calibrated probabilities matter.

### Q164. What is model calibration and how do you calibrate?

A model is calibrated if, among predictions with probability 0.8, about 80% are positive. Check with reliability diagrams, Brier score, Expected Calibration Error (ECE). Calibrate with Platt scaling (logistic fit on scores), isotonic regression, or temperature scaling (neural nets). Boosted trees, SVMs, Naive Bayes, and deep nets are often miscalibrated.

### Q165. How do you quantify uncertainty in predictions? What is conformal prediction?

**Aleatoric** uncertainty comes from noise in the data (irreducible); **epistemic** uncertainty comes from limited knowledge (reducible with more data). Methods: calibrated probabilities, prediction intervals via quantile regression, ensembles (spread of predictions), MC dropout, Bayesian models. **Conformal prediction** uses a held-out calibration set of residuals/non-conformity scores to turn any model's output into prediction intervals (regression) or prediction sets (classification) with a guaranteed coverage such as 90%, assuming exchangeable data.

### Q166. What are ranking metrics?

**Precision@k / Recall@k**; **MAP** (mean average precision over queries); **MRR** (mean of 1/rank of first relevant item); **NDCG@k** (discounted cumulative gain with graded relevance, normalized by ideal ordering); hit rate; coverage and diversity for recommenders.

### Q167. What are learning curves and validation curves?

**Learning curves** plot train/validation score vs training set size: a large gap means high variance (more data helps); both low and close means high bias. **Validation curves** plot scores vs a hyperparameter value to find the sweet spot between under- and overfitting.

### Q168. How do you compare two models statistically?

Use the same CV folds and a paired test: paired t-test on fold scores (corrected resampled t-test), McNemar's test on classification disagreements, 5×2 CV test, or bootstrap confidence intervals of the metric difference.

### Q169. What metrics are used for object detection, segmentation and NLP generation?

Detection: IoU, mAP@0.5, mAP@[0.5:0.95]. Segmentation: pixel accuracy, mean IoU, Dice coefficient. Generation: BLEU (n-gram precision, translation), ROUGE (recall, summarization), METEOR, BERTScore, perplexity, human/LLM-as-judge evaluation.

### Q170. How do you explain a model's predictions (feature importance, SHAP, LIME)?

**Global** explanations describe overall behavior: coefficients (linear models), permutation importance (drop in validation score when a feature is shuffled), partial dependence plots. **Local** explanations describe one prediction: **SHAP** assigns each feature its Shapley value (average marginal contribution across feature combinations; contributions sum to prediction minus baseline; TreeSHAP is fast for tree models); **LIME** fits a simple surrogate model on perturbed samples around the instance. Needed for trust, debugging, leakage detection and regulatory compliance.

## 10. Model Selection & Hyperparameter Tuning

### Q171. Grid search vs random search vs Bayesian optimization?

**Grid search**: exhaustive over a grid; expensive, scales exponentially. **Random search**: samples combinations; more efficient since few hyperparameters usually matter (Bergstra & Bengio). **Bayesian optimization**: builds a surrogate model (Gaussian process, TPE) of score vs hyperparameters and picks the next trial using an acquisition function (Expected Improvement, UCB), balancing exploration/exploitation. Tools: Optuna, Hyperopt, Ray Tune.

### Q172. What are AIC and BIC?

Model selection criteria balancing fit and complexity: AIC = $2k - 2 \ln L$; BIC = $k \ln n - 2 \ln L$. Lower is better; BIC penalizes complexity more and prefers simpler models for large n.

### Q173. What is Occam's razor in ML?

Among models with similar performance, prefer the simpler one: it generalizes better, is easier to interpret, maintain and serve.

### Q174. Which hyperparameters matter most for common models?

Random forest: n_estimators, max_features, max_depth, min_samples_leaf. GBM/XGBoost: learning_rate, n_estimators (with early stopping), max_depth/num_leaves, subsample, colsample_bytree, min_child_weight, regularization. SVM: C, kernel, γ. Neural nets: learning rate (most important), batch size, architecture size, weight decay, dropout, optimizer, schedule.

## 11. Time Series

### Q175. What are the components of a time series?

Trend (long-term direction), seasonality (fixed-period patterns), cyclical (non-fixed longer cycles), and residual/noise. Decompose additively ($y = T + S + R$) or multiplicatively ($y = T \times S \times R$), e.g., STL decomposition.

### Q176. What is stationarity and how do you test/achieve it?

A stationary series has constant mean, variance and autocovariance over time. Tests: Augmented Dickey-Fuller (H0: non-stationary/unit root), KPSS (H0: stationary). Achieve via differencing, seasonal differencing, log/Box-Cox transforms, detrending.

### Q177. Explain ACF and PACF.

ACF: correlation of the series with its lags (including indirect effects). PACF: correlation with a lag after removing effects of shorter lags. For AR(p), PACF cuts off after p; for MA(q), ACF cuts off after q.

### Q178. Explain ARIMA and SARIMA.

ARIMA(p, d, q): AutoRegressive terms (p past values), Integrated (d differences), Moving Average (q past errors). SARIMA adds seasonal (P, D, Q, s) terms; ARIMAX/SARIMAX adds exogenous variables. Select orders via ACF/PACF, AIC, or auto-ARIMA.

### Q179. What other forecasting methods exist?

Naive / seasonal naive baselines, moving averages, exponential smoothing (Holt-Winters: level, trend, seasonality), Prophet (trend + seasonality + holidays), VAR (multivariate), gradient boosting with lag/rolling features, RNN/LSTM, Temporal Convolutional Networks, Transformers (Temporal Fusion Transformer, Informer, PatchTST), N-BEATS, foundation models (TimeGPT, Chronos).

### Q180. How do you validate time series models?

Never shuffle. Use walk-forward / expanding or sliding window validation (TimeSeriesSplit), with a gap between train and test if features use lags. Compute features only from past data. Metrics: MAE, RMSE, MAPE, sMAPE, MASE (scaled by naive forecast).

### Q181. How do you convert a time series into a supervised learning problem?

Create lag features ($y_{t-1}$, $y_{t-7}$), rolling statistics (mean, std over windows, shifted to avoid leakage), date features (day, month, holiday), and exogenous variables. For multi-step forecasting: recursive (feed predictions back), direct (one model per horizon), or multi-output.

## 12. Recommender Systems

### Q182. What are the main types of recommender systems?

**Content-based**: recommend items similar to those the user liked using item features. **Collaborative filtering**: use user-item interactions; user-based (similar users) or item-based (similar items). **Hybrid**: combine both. **Knowledge-based** and **popularity-based** baselines.

### Q183. Explain matrix factorization for recommendations.

Approximate the sparse user-item rating matrix $R \approx U \cdot V^T$ with low-dimensional latent factors for users and items, learned by minimizing squared error on observed entries plus regularization, via SGD or Alternating Least Squares (ALS). Predicted rating = $u_i \cdot v_j$ + biases. Popularized by the Netflix Prize (SVD++, etc.).

### Q184. Explicit vs implicit feedback?

Explicit: ratings, likes (sparse, biased). Implicit: clicks, views, purchases, watch time (abundant, noisy, no negatives). Implicit needs negative sampling and losses like BPR (Bayesian Personalized Ranking) or weighted ALS.

### Q185. What is the cold-start problem and how to address it?

New users or items lack interaction history. Solutions: content-based features, popularity/trending recommendations, onboarding questionnaires, demographic/contextual features, hybrid models, exploration via bandits, and using metadata embeddings.

### Q186. Describe a modern large-scale recommender architecture.

Multi-stage: **candidate generation** (two-tower models producing user and item embeddings, retrieved via approximate nearest neighbor search) → **ranking** (heavier model such as Wide & Deep, DeepFM, DCN, DLRM, or GBDT with rich features predicting click/engagement) → **re-ranking** (diversity, freshness, business rules, fairness). Evaluate offline (NDCG, recall@k) and online (A/B: CTR, retention).

### Q187. What is learning to rank? Pointwise vs pairwise vs listwise?

Training models to order items by relevance for a query or user. **Pointwise**: predict a relevance score/label per item (regression/classification). **Pairwise**: learn to order pairs correctly (RankNet, RankSVM). **Listwise**: optimize a metric over the whole list such as NDCG (ListNet, LambdaRank). **LambdaMART** (gradient boosted trees with lambda gradients; `rank:ndcg` in XGBoost, `lambdarank` in LightGBM) is a strong industry standard. Click data needs position-bias correction.

### Q188. What are multi-armed bandits?

Online methods balancing exploration and exploitation: ε-greedy, Upper Confidence Bound (UCB), Thompson sampling. Contextual bandits (LinUCB) use features. Used for recommendations, ad selection and adaptive A/B testing.